

Reaching CAS(16,16)

Architectural priorities for the SlowQuant state-vector code

Prepared 19 September 2026 | branch `alpha_beta_speedup` at c47ea1c | all figures measured single-threaded on N2/6-31G unless marked as a projection

The short version. CAS(16,16) is 165,636,900 determinants and a CI vector of 1.23 GB. Nothing about the physics requires more than a few of those vectors. As the code stands the supporting structures would need **more than 100 GB**, and the largest single item is not the determinant index but the rotation layouts added in this session, at roughly 70 GB. Every one of those structures is $O(N_{\text{det}})$ where the alpha/beta product makes $O(N_{\text{alpha}} + N_{\text{beta}})$ sufficient, a factor of about 6400. The work below is mostly about spending memory in string space instead of determinant space; it happens to make the kernels faster too.

1. Where the memory goes

The table projects each structure to CAS(16,16) from measurements at CAS(8,8) through CAS(14,14). Bytes per determinant are stable across sizes for everything except the Numba dictionary, whose table doubles in powers of two, so its entry is given as a range.

Structure	Scales as	At CAS(12,12)	Projected at CAS(16,16)
Rotation layouts, all generators	N_{det} x generators	300 MB	about 70 GB (240 generators)
det2idx, Numba typed dictionary	N_{det}	151 MB	12 to 16 GB
Python list and dict during construction	N_{det}	97 MB	about 22 GB (transient)
idx2det	N_{det}	6.5 MB	1.23 GB
CI vectors alive at once (about five)	N_{det}	33 MB	about 6 GB
Per-spin excitation maps (spin arena)	N_{alpha}	8.6 MB	0.12 GB
Two-electron density matrix	n^4	0.2 MB	0.5 MB

The three highlighted rows are the whole problem. They are also the three that the alpha/beta product structure makes unnecessary. The rotation layout figure deserves emphasis because it is new: it is a consequence of caching, per generator, a list of the determinant pairs that generator rotates. That list is about 21 per cent of the CI space whatever the size, so it costs roughly 294 MB per generator at CAS(16,16) regardless of how the ansatz is chosen. The ansatz only decides how many times that is paid.

Ansatz	Generators at (12,12)	Generators at (16,16)	Layout cache at (16,16)
ksafupccgsd, 1 layer	132	240	about 70 GB
SAfUCCSD	702	2,144	about 630 GB
fUCCSD	1,818	5,792	about 1.7 TB

What the target looks like afterwards. With the changes in sections 2 and 3, CAS(16,16) needs about 6 GB of CI vectors, 1.2 GB of idx2det if it is kept at all, and roughly 15 MB of layouts. Call it **7 to 8 GB**, which fits on a workstation.

2. Prioritized changes

Ordered by benefit per unit of disruption. P0 to P2 are what make CAS(16,16) possible at all; P3 onwards are speed once it runs.

P0 Hold the rotation and block layouts in string space, not determinant space

Gain	About 70 GB down to about 15 MB at CAS(16,16), a factor of roughly 4600. Also 1.5 to 4.8 times faster kernels.
Cost	Moderate. Needs the per-generator spin decomposition that the determinant-pair form was chosen to avoid.
Evidence	Measured 294 MB per generator at CAS(8,8), (10,10) and (12,12) alike; 368 bytes per determinant. The string-pair kernel was timed at 408 us against 633 us for the determinant-pair kernel on a spin-adapted single at CAS(12,12).

A pure-alpha excitation changes only the alpha string, so the pairs it rotates are pairs of alpha strings, not pairs of determinants. At CAS(16,16) that is about 3,400 pairs instead of 44.7 million, and the same 2x2 rotation then applies to a whole row of the CI matrix at once.

I chose the determinant-pair form deliberately, because it needs no per-excitation-type spin analysis and works unchanged on a space that is not a product. That was the right call for CAS(12,12), where the cache is 300 MB. It is the wrong call at CAS(16,16). This is the single most important item in this document, and it is a direct consequence of work committed in this session.

P1 Stop storing det2idx for product spaces

Gain	12 to 16 GB down to zero.
Cost	Low. The lookup is two small dictionary probes and one multiply.
Evidence	The general kernel, the only real consumer, was called zero times across ksafupccgsd, fUCCSD and SAFUCCSD evaluations on a product space. The only other use is a single lookup when building the reference state.

Over a product space the index of a determinant is $\text{idx}_{\alpha} \times N_{\beta} + \text{idx}_{\beta}$, so the determinant-to-index map is redundant with the two per-spin maps, which hold 12,870 entries each rather than 165,636,900.

The map still has to exist for the extended space used by the self-consistent linear response, which is not a product. That is exactly the two-track split in section 3.

P2 Build the CI space without Python containers

Gain	About 22 GB of transient peak removed, and most of the construction time.
Cost	Low, and contained to one function.
Evidence	Measured 131 bytes per determinant in the Python list and dict at CAS(14,14), against 8 bytes for the numpy array they become. Construction took 27.6 s at CAS(14,14); CAS(16,16) is fourteen times larger.

get_indexing appends to a Python list and fills a Python dict, then CI_Info converts the dict to a Numba typed dictionary while the Python one is still alive. The counts are known in advance from binomial coefficients, so both can be written straight into preallocated arrays.

This matters even if P1 is not done, because the transient peak is what decides whether the job starts at all.

P3 Derive idx2det rather than storing it

Gain	1.23 GB.
Cost	Low, but it touches every kernel signature.
Evidence	8 bytes per determinant, measured; identical in size to a CI vector.

A determinant is recoverable from its row and column in the CI matrix. Worth doing only after P0 to P2, since on its own it saves about as much as one CI vector.

P4 Apply the derivative generator through the layout in get_grad_action

Gain	6.2 times on a step that is 17 per cent of an evaluation.
Cost	Low. The layout is already built and cached for the matching unitary.
Evidence	Measured at CAS(12,12): 3,665 us as it stands against 593 us through a kernel using the pairing, or 14.5 passes over the CI vector against 2.3.

get_grad_action still applies the generator through the general operator machinery even though the same generator's pairing is sitting in the cache.

P5 Rotate in place where the buffer is already private

Gain	About 10 per cent of an evaluation.
Cost	Low, one extra argument.
Evidence	990 calls per evaluation at CAS(12,12), each copying the full vector, at 250 us per copy.

Every application of exp(theta T) copies the state before rotating. construct_ups_state already owns a private working vector, so its share of those copies is pure waste.

P6 Build the layouts more cheaply

Gain	Cold start at CAS(12,12) from 3.35 s towards 0.4 s.
Cost	Low.
Evidence	Measured 25.4 ms per generator, which is 99 passes over the CI vector: two probes at 10 passes, flatnonzero 8.4, allclose 6.7, the verification sweep 25.9, and about 48 in intermediate arrays.

The information a layout needs is worth about ten passes. Most of the rest is full-length numpy temporaries. This becomes moot for product spaces once P0 lands, since string-space layouts are thousands of times smaller to build.

P7 Thread the sigma3 contraction

Gain	Up to the core count on 24 per cent of an evaluation.
Cost	Moderate, and interacts with the BLAS thread policy.
Evidence	Measured at CAS(12,12): the two Hamiltonian applications take 0.59 s of a 2.45 s evaluation, all of it in one kernel.

After P0 to P6 the Hamiltonian contraction is the largest remaining single item, and it is the part that most resembles a conventional direct CI sigma build.

3. The two tracks

You asked for two tracks in the operator-state algebra, and the analysis above supports that strongly. What exists today is not two tracks: it is one determinant-indexed design with a single guard on `CI_Info.is_spin_product` buried inside `factorize_operator`. Everything else, including all four caches on `CI_Info` and every kernel signature, is shaped for the determinant-indexed view. That is why the product structure buys speed today but no memory.

	Track A: alpha/beta product	Track B: general space
Produced by	<code>get_indexing</code>	<code>get_indexing_extended</code> , and the generalized wave function later
Indexing	$I = I_{\text{alpha}} \times N_{\text{beta}} + I_{\text{beta}}$	explicit determinant to index map
Index memory	$O(N_{\text{alpha}} + N_{\text{beta}})$	$O(N_{\text{det}})$
Operator form	factorizes per spin	general fermionic string
Layout memory per generator	$O(N_{\text{alpha}})$, about 31 KB at (16,16)	$O(N_{\text{det}})$, about 294 MB at (16,16)
Kernels	row and column operations on the CI matrix; GEMM for the contraction	gather and scatter over determinants
Caching policy	cache freely, it is small	build lazily, do not cache, or cache to a budget

The practical shape of this is a common `CI_Info` base with two subclasses owning their own index maps and their own caches, and one dispatch at the top of the operator-state algebra rather than deep inside it. Track B keeps today's behaviour; the sizes it is used at are much smaller, so $O(N_{\text{det}})$ layouts are tolerable there, and not caching them at all is probably right.

4. Does the generalized wave function keep the product structure?

Partly, and the answer turns on S_z rather than on S^2 . The Fock space over $2n$ spin orbitals factorizes as F_{alpha} tensor F_{beta} whenever each orbital carries a definite spin label. Three cases follow, and they behave quite differently.

Case	Orbitals	Determinant space	Track
S^2 broken, S_z good (UHF-like, unequal alpha and beta counts)	carry a spin label	a single alpha x beta rectangle: an exact Cartesian product	A, and already supported
S_z broken, orbitals still labelled alpha or beta	carry a spin label	a union of rectangles, one per $(N_{\text{alpha}}, N_{\text{beta}})$ sector with the total fixed	A, graded by sector
Generalized with spinor orbitals (GHF-like), and two- or four-component	mix alpha and beta, no label	no factorization of any kind	B

So the correction you invited is this. Breaking S^2 on its own costs nothing: `get_indexing` already takes the alpha and beta electron counts separately, and an unrestricted reference is a product space like any other. Breaking S_z costs the single-rectangle indexing but not the factorization itself: the space becomes a union of product blocks, indexed by (sector, $I_{\text{alpha}}, I_{\text{beta}}$), with per-spin algebra still valid inside a sector and S_z -changing operators

mapping between sectors. That is a generalization of today's indexing rather than an abandonment of it, and most of the memory argument in this document survives it.

Two- and four-component wave functions are the genuine Track B case. Once the molecular orbitals are spinors mixing alpha and beta character, no orbital carries a spin label, so there is no bipartition to factorize over and no per-spin string algebra to be had. Four-component adds the positronic branch on top. For those, the determinant-indexed machinery is the right and only answer, which is a further argument for keeping Track B genuinely general rather than treating it as a fallback that happens to work.

5. What about time?

Memory is the wall, but it is worth being honest about the other side. A warm energy and gradient evaluation at CAS(12,12) with ksafupccgsd takes 2.45 s today. CAS(16,16) is 194 times more determinants, and most of the remaining cost is linear in that, so a single evaluation should be expected in the region of **eight to twenty minutes** single-threaded, the spread reflecting the contraction scaling worse than linearly. A wave function optimization needing a hundred evaluations is therefore a multi-day run before any threading. P4, P5 and P7 together are worth roughly a factor of two on that, and P7 is worth the core count on the largest single piece. This paragraph is an extrapolation, not a measurement.

6. Questions

Answers to these would change the ordering above, in some cases substantially.

1. Does the generalized branch break Sz, and do its orbitals keep an alpha or beta label?

This decides whether it is Track A graded by sector, which preserves most of the memory argument, or genuinely Track B. Section 4 sets out the three cases.

2. What memory budget should CAS(16,16) fit in?

The plan above targets 7 to 8 GB. If the target is 128 GB then P3 and parts of P2 are not worth doing; if it is 16 GB then P0 and P1 are both mandatory and idx2det has to go as well.

3. Which ansatz at CAS(16,16), and with orbital optimization or theta only?

ksafupccgsd needs 240 generators, fUCCSD 5,792. That is a factor of 24 on the layout memory and on the per-evaluation cost, and it decides whether P0 alone is sufficient.

4. Is the extended space ever going to be used at these sizes?

If the self-consistent linear response is only ever run at CAS(12,12) and below, Track B can stay simple and uncached. If it has to reach CAS(16,16), it needs its own memory plan.

Appendix: where the numbers come from

Everything labelled as measured was taken on this machine, single-threaded with OMP, OpenBLAS and MKL thread counts pinned to one, on N2/6-31G at the active space stated, on branch alpha_beta_speedup at c47ea1c. Per-determinant costs were taken at three or four active space sizes and were stable, which is what licenses the projections to CAS(16,16); the exception is the Numba typed dictionary, whose table size doubles in powers of two, so its cost per determinant oscillates between about 65 and 190 bytes and is quoted as a range. Determinant counts are exact binomial coefficients. The CAS(16,16) figures are projections throughout, since the space has not been built: CAS(14,14) at 11.8 million determinants is the largest actually constructed here, and it took 27.6 s and 2.64 GB.

One caveat worth repeating. P0 exists because of a decision I made earlier in this session. The determinant-pair layout was chosen for generality and simplicity, and it was measured and validated at CAS(10,10) and CAS(12,12), where it is unambiguously the right shape. Its memory cost per generator is constant in bytes per determinant, so it was invisible at those sizes and becomes fatal at CAS(16,16). The lesson is that a cache sized as a fraction of the CI space needs to be justified against the largest space intended, not the one in front of you.